

Swarmbly AI

Una inteligencia artificial construida entre todos

Documento divulgativo Sebastián A. Espinoza-Ulloa, Ph.D. · 14 de agosto de 2026 · actualizado el 29 de septiembre de 2026

Lo esencial en un párrafo

Hoy, para usar inteligencia artificial avanzada, dependemos de un puñado de empresas que poseen enormes centros de datos. Swarmbly AI propone otra cosa: que la IA funcione repartiendo el trabajo entre los computadores que ya tenemos en casa y en la oficina, la mayoría de los cuales pasa el día encendida sin hacer casi nada. La idea técnica que lo hace posible viene de un lugar inesperado: de cómo los biólogos reconstruyen un genoma completo a partir de miles de fragmentos pequeños de ADN.

El nombre, en un párrafo

Swarmbly son dos palabras en inglés: *swarm* (enjambre) y *assembly* (ensamblaje). El enjambre es la mitad fácil: conseguir que muchos computadores trabajen a la vez es un problema resuelto hace décadas. El ensamblaje es la mitad difícil, y es donde este proyecto puede fracasar: un enjambre no produce una respuesta, produce *fragmentos*, y convertir muchos fragmentos independientes en un texto que una persona pueda leer sin notar las junturas es la dificultad entera. Esa segunda mitad viene prestada de la genética, donde leer un cromosoma de corrido es imposible, así que se lo rompe en millones de pedazos, se lee cada uno por separado y se reconstruye el original a partir de los solapamientos. El nombre dice lo que el sistema hace —enjambra y ensambla— y deliberadamente no dice de dónde salió la idea, porque eso corresponde al artículo técnico, donde puede enunciarse con precisión.

1. El problema no es el conocimiento. Es el dinero.

Existe una confusión muy extendida sobre por qué la inteligencia artificial está concentrada en tan pocas manos. Mucha gente supone que las grandes empresas guardan un secreto: una fórmula, un algoritmo que nadie más conoce.

No es así. Buena parte de los modelos de IA se publican abiertamente. Cualquiera puede descargarlos. Los métodos se explican en artículos científicos que están al alcance de todos. El conocimiento circula.

Lo que no circula es el dinero para **operarlos**. Poner un modelo grande a responder preguntas para millones de personas exige aceleradores gráficos que cuestan decenas de miles de dólares cada uno, edificios enteros para alojarlos, contratos de electricidad del tamaño de los de una ciudad pequeña, y redes internas de una velocidad que no existe fuera de esas instalaciones.

Ahí está la concentración real. Una tecnología cuyo **conocimiento** es público termina siendo controlada por quien puede pagar el **hardware**.

Las cifras dan la escala del asunto. Los centros de datos consumieron cerca de 415 teravatios-hora de electricidad en 2024, aproximadamente el 1.5 % de toda la electricidad del mundo, y las proyecciones los sitúan cerca de 945 para 2030. Los grandes centros estadounidenses se alimentan además de redes eléctricas más sucias que el promedio nacional: unos 545 gramos de CO₂ por kilovatio-hora frente a 370.

Es el retrato de una industria cuyo camino de crecimiento pasa por construir. Y construir solo está al alcance de quien puede financiarlo.

2. Y sin embargo, el hardware ya existe

Aquí está el dato que da sentido a todo el proyecto.

Existe una comunidad de personas que desde hace décadas presta el tiempo libre de sus computadores a proyectos científicos —búsqueda de señales de radio, plegamiento de proteínas, modelos climáticos—. Ese sistema, llamado BOINC, reúne hoy alrededor de **700000 dispositivos activos, cuatro millones de núcleos de procesador y 560000 tarjetas gráficas**, con una potencia media de 93 petaflops.

Ahora, lo importante: esa cifra proviene de una comunidad que **ha ido menguando** durante veinte años, de cerca de un millón de voluntarios a unos doscientos mil. Es decir, no es un techo. Es un piso, medido en un nicho en retroceso.

Y por debajo de esos voluntarios organizados hay cientos de millones de computadores personales, portátiles y equipos de oficina que encienden su tarjeta gráfica un rato al día y la dejan ociosa el resto. Ese hardware ya se fabricó. Ya consume electricidad al estar encendido. Ya está pagado.

La capacidad de cómputo sobrante del mundo no es una hipótesis. Existe. Lo que falta es una forma de usarla.

3. Por qué nadie lo ha logrado todavía

Ha habido intentos serios, y el más conocido —un proyecto llamado Petals— merece reconocimiento: demostró que una IA repartida entre voluntarios por internet es posible. Sin él, esta propuesta no tendría sobre qué apoyarse.

Pero todos esos intentos comparten una decisión de diseño que los limita: **reparten el modelo**.

Imagina que el modelo de IA es un cerebro enorme. Estos sistemas lo cortan en rebanadas y le dan una rebanada a cada participante. Para producir **una sola palabra** de la respuesta, el pensamiento tiene que viajar por internet desde la primera rebanada a la segunda, de la segunda a la tercera, y así hasta el final. Y luego vuelve a empezar para la palabra siguiente.

El problema es la distancia. Dentro de un centro de datos, las tarjetas gráficas se hablan a unos 900 gigabytes por segundo. La conexión de subida de una casa típica ronda los 60 megabits por segundo. La diferencia es de aproximadamente **120000 veces**. En tiempo de respuesta la brecha es de cuatro a cinco órdenes de magnitud: microsegundos dentro del centro de datos, decenas o cientos de milisegundos entre ciudades.

Cruzar ese abismo **una vez por cada palabra** es insostenible. Y las mediciones lo confirman: Petals pierde alrededor de un 31 % de su rendimiento solo por pasar de una conexión de

centro de datos a una conexión doméstica.

No es que lo hayan hecho mal. Es que están respondiendo bien a la pregunta equivocada.

4. La idea de Swarmbly: repartir el problema, no el cerebro

Swarmbly cambia la pregunta. En lugar de «¿cómo hacemos funcionar un cerebro gigante entre muchas máquinas?», pregunta:

¿Cómo ponemos muchos cerebros pequeños y completos a trabajar sobre un problema grande?

Cada participante ejecuta un modelo **entero**, pequeño pero completo y autónomo, del tamaño que cabe holgadamente en un computador normal. Lo que se reparte no es el cerebro: es **la tarea**.

Y como cada participante recibe un encargo completo y devuelve una respuesta completa, **la información cruza internet una sola vez por encargo**, no una vez por palabra. Ese cambio es lo que saca al sistema del abismo del apartado anterior. No es una mejora sobre lo anterior: es un régimen distinto.

5. De dónde viene la idea: cómo se lee un genoma

Aquí es donde el proyecto toma prestada su idea central de la biología, y conviene contarla bien porque es el corazón de todo.

Un genoma humano tiene unos tres mil millones de letras. Ninguna máquina puede leerlo de corrido. Lo que hacen los laboratorios desde hace décadas es algo que parece absurdo la primera vez que se escucha: **rompen el ADN en millones de trozos pequeños al azar**, leen cada trocito por separado, y después reconstruyen la secuencia completa a partir de ese montón de fragmentos.

Funciona por dos razones.

La primera es la **redundancia**. No se lee cada zona una vez, sino muchas veces. Cada punto del genoma aparece en varios fragmentos distintos, y esa repetición es lo que permite corregir errores: si nueve lecturas dicen una cosa y una dice otra, la discrepante era un error de lectura.

La segunda es el **solapamiento**. Los fragmentos se superponen en sus bordes, y ese solape es lo que indica cómo encajan entre sí, igual que en un rompecabezas.

Swarmbly aplica exactamente esa lógica al lenguaje.

En genómica	En Swarmbly
Un genoma que hay que leer	Una tarea que hay que resolver
Fragmentos de ADN (<i>reads</i>)	Micro-tareas enviadas a distintos computadores
Solapamiento entre fragmentos	Contexto compartido entre micro-tareas
Reconstrucción del genoma	Ensamblaje de la respuesta en tu computador
Zonas de baja calidad marcadas	Partes de la respuesta poco fiables, señaladas

Y hay un motivo por el que esta analogía no es decorativa: el autor de la propuesta es doctor en genómica de poblaciones y trabaja profesionalmente ensamblando genomas. La idea no

viene de una metáfora tomada de fuera. Viene de dentro del oficio.

6. Cómo funciona, paso a paso

Supongamos que le pides al sistema un informe extenso sobre un tema.

Primero, tu computador decide si vale la pena repartir. Un pequeño modelo que corre localmente evalúa la petición. Si preguntas algo simple y directo —«¿qué temperatura hace hoy?»— **no** la reparte: la resuelve de una vez. Esto importa más de lo que parece, y lo explico en el apartado siguiente.

Segundo, si la tarea es divisible, la planifica. Divide el trabajo en partes con sentido propio —introducción, antecedentes, análisis, conclusiones— y determina qué partes dependen de otras y cuáles pueden hacerse a la vez.

Tercero, redacta un contrato común. Antes de enviar nada, escribe una nota breve que acompaña a *todos* los encargos: cuál es el objetivo, para quién es, en qué tono, en qué formato, cómo se llama cada cosa. Es lo que evita que una parte llegue escrita como un artículo académico y la siguiente como una entrada de blog.

Cuarto, reparte. Cada micro-tarea viaja a un computador voluntario distinto. Y cuando la parte es importante, se envía **la misma tarea a varios participantes a la vez**, eligiendo deliberadamente equipos que ejecutan **modelos de familias diferentes**.

Quinto, verifica. El sistema comprueba que cada participante usó realmente el modelo que declaró y no uno más pequeño y barato. Existen métodos que hacen esta comprobación con una fiabilidad muy alta y un coste casi nulo.

Sexto, ensambla. Tu computador junta las piezas. Cuando hay varias respuestas a la misma tarea, las **alineas entre sí** —igual que un biólogo alinea lecturas de ADN— y produce una versión de consenso.

Y séptimo, te dice de qué se fía. Esto es lo que viene a continuación, y es lo más interesante del proyecto.

7. El mapa de confianza: algo que la IA centralizada no puede darte

Cuando un modelo de IA te responde hoy, te entrega un texto que suena igual de seguro de principio a fin. Las partes que sabe bien y las que se está inventando tienen exactamente el mismo tono.

En Swarmbly, como la misma tarea la resolvieron varios modelos **distintos e independientes**, sus respuestas se pueden comparar entre sí. Y esa comparación dice algo:

- Donde varios modelos que no tienen nada que ver entre sí **coinciden**, hay una convergencia que significa algo.
- Donde **discrepan**, hay una señal clara de que ese punto merece que lo revises.

El sistema entrega entonces, junto a la respuesta, **un mapa de las zonas poco fiables**. Igual que un ensamblador de genomas no entrega una secuencia uniformemente confiada, sino que marca las regiones donde la lectura fue dudosa.

Y aquí está lo notable: **un proveedor centralizado no puede ofrecer esto**, por mucho dinero que tenga. Con un solo modelo no hay nada contra qué comparar. Preguntarle diez veces a un mismo modelo mide su indecisión, no el desacuerdo entre observadores independientes.

La redundancia que la descentralización **necesita** resulta ser exactamente lo que produce esta información. Un costo de la arquitectura y una capacidad de ella son el mismo mecanismo visto desde dos lados.

Con una advertencia honesta: **que varios modelos coincidan no garantiza que tengan razón**. Si todos aprendieron del mismo material erróneo, pueden equivocarse juntos. Por eso el diseño insiste en usar modelos de familias distintas, y por eso la relación entre «coinciden» y «es correcto» está planteada como algo **a medir**, no como algo que se dé por hecho. Si al medirla resulta que no se sostiene, habrá que decirlo.

Ya se ha medido, cuatro veces, y la relación no se sostuvo. Así que se dice: la afirmación de que el acuerdo predice la corrección queda **retirada** — véase el apartado 11. De este apartado sobrevive el mecanismo y nada más. El sistema todavía puede decirte dónde discreparon modelos independientes, y eso sigue siendo algo que un único modelo centralizado no puede hacer; lo que no puede decirte es que los lugares donde coincidieron sean los lugares donde acertó. Lee el resto de este apartado con esa corrección aplicada.

8. Por qué el sistema a veces se niega a repartir

Este punto merece su propio apartado porque es contraintuitivo, y porque entenderlo es entender los límites reales de la propuesta.

Repartir no siempre ayuda. Hay peticiones que **no se pueden** dividir sin destruirlas.

Piensa en «¿qué temperatura hace hoy en Saskatoon?». Alguien podría pensar en trocearla: mandar «¿qué temperatura hace?» a un computador, «¿hoy?» a otro, «¿en Saskatoon?» a un tercero, y juntar las respuestas.

No funciona, y la razón es exacta. El que recibe «¿qué temperatura hace?» devolverá una temperatura de ningún lugar y de ningún momento. No es una respuesta parcialmente correcta que luego se pueda completar: es una respuesta a otra pregunta.

En términos genómicos: un fragmento de ADN es un **trozo contiguo y completo** de la molécula. Trocear esa pregunta no es fragmentar en trozos — es tomar un fragmento y **borrarle una de cada tres letras**. Ninguna cantidad de redundancia recupera eso, porque no es muestreo: es información destruida antes de muestrear.

Por eso Swarmbly reparte en dos niveles distintos:

- **Nivel macro**, para tareas grandes que sí tienen partes con sentido propio: un informe con secciones, un lote de mil documentos.
- **Nivel micro**, donde se envía **la tarea completa** a varios participantes y se comparan sus respuestas.

Una pregunta atómica salta directamente al nivel micro. Y un sistema que trocee preguntas atómicas está, según la especificación del protocolo, **funcionando mal**.

9. Tres círculos: adónde va realmente tu información

Los apartados anteriores describen *qué* hace Swarmbly con una petición. Este describe *adónde* va, porque son preguntas distintas y confundirlas es la forma habitual de romper una promesa de privacidad.

Antes de repartir nada, un pequeño programa **en tu propia máquina** lee la petición y decide a cuál de tres círculos pertenece. Ese programa no le pregunta nunca nada a la red: una comprobación de privacidad que llama a un servidor para preguntar si tu texto es privado ya ha entregado tu texto.

El círculo exterior: la red abierta. Las peticiones corrientes —una entrada de blog, el resumen de un artículo público, código que ibas a publicar de todos modos— van a la malla de voluntarios descrita hasta aquí. Muchas máquinas desconocidas, redundancia completa, verificación completa.

El círculo intermedio: un enjambre de confianza. Una empresa, un hospital, una universidad o un grupo de investigación pueden levantar su propia malla cerrada. Sólo pueden entrar las máquinas de una lista explícita, cada conexión demuestra la identidad de *ambos* extremos, y el conjunto suele vivir en la red de la propia organización. Es el mismo programa y el mismo protocolo que el círculo exterior: lo único que cambia es quién puede entrar.

Esta es la diferencia entre «no puede usarse con historias clínicas» y «puede usarse con historias clínicas, en las máquinas del propio hospital». La normativa de protección de datos está escrita alrededor de organizaciones a las que se puede nombrar y exigir responsabilidades; un voluntario anónimo no puede serlo nunca, y una máquina de la lista del hospital lo es siempre.

El círculo interior: sólo tu máquina. Para el material más sensible no sale absolutamente nada. Tu computador hace el trabajo entero por sí solo. Es menos capaz, y ese es el precio honesto de la garantía; es también el único caso en el que el proyecto hace una promesa absoluta sobre el secreto, y puede hacerla únicamente porque no hay red de por medio.

Puedes elegir el círculo a mano, y tu elección manda siempre. Si no has elegido, la comprobación local peca de prudente: cuando ve algo que parece una historia clínica, una contraseña, una cuenta bancaria o un expediente judicial, mueve la petición hacia dentro por su cuenta. Es deliberadamente exagerada, porque mandar una petición inofensiva a un círculo cerrado cuesta algo de velocidad mientras que el error contrario cuesta justo aquello que el círculo existe para proteger.

Una pega honesta, que conviene decir sin rodeos. En un círculo cerrado se puede encargar cada tarea a una sola máquina en lugar de a varias, porque las máquinas son conocidas y no hay motivo para esperar que ninguna mienta. Pero comparar varias respuestas es exactamente como se construye el mapa de confianza del apartado 7. Preguntar a una sola ahorra tiempo y pierde el mapa. El sistema no hace esto en silencio: cuando ocurre, la respuesta vuelve diciendo explícitamente que no se produjo mapa de confianza. Elegir entre velocidad y saber qué conviene comprobar es una decisión real, y le corresponde a quien opera la red, no al programa.

Y un círculo cerrado no elimina la confianza: la traslada. Quien decide quién está en la lista tiene ese poder, y una máquina comprometida *dentro* del círculo es más peligrosa que una máquina desconocida fuera de él, precisamente porque se están comparando menos respuestas. El proyecto lo dice en su propia especificación en lugar de dejar que se descubra.

10. Qué gana la gente

Usar IA sin poder comprar el hardware. Para quien tiene una tarjeta gráfica potente, existen técnicas que aceleran los modelos localmente. Para quien no la tiene, esas técnicas no sirven de nada. La comparación relevante no es «más rápido o más lento», sino «**puedo hacerlo o no puedo**». Decir que el autobús es más lento que el coche solo es un argumento si tienes coche.

Capacidad que crece con la gente, no con el capital. Cada persona que se suma aporta capacidad. Es una curva de crecimiento que ninguna empresa puede igualar, porque la suya está limitada por lo que pueda construir y financiar.

Tareas que hoy son inviables por su tamaño. Procesar diez mil documentos, revisar un archivo entero, analizar un corpus completo. Trabajos que se cobran caros precisamente por su volumen, y que aquí se reparten entre muchos.

Saber de qué fiarse. El mapa de confianza del apartado 7 — un mecanismo que funciona, pero cuya utilidad las mediciones han **retirado** en lugar de confirmar (apartado 11). Se nombra aquí en vez de retirarlo en silencio, porque este documento lo anunció antes de que estuviera probado.

Un sistema que se puede auditar en vez de crear. El protocolo es público, el programa es público, y el sistema informa por sí mismo de cuánta calidad perdió al repartir. No como cortesía: como requisito de diseño.

Aprovechar lo ya fabricado. La mayor parte de la huella ambiental de la IA no está en la electricidad que consume funcionando, sino en **fabricar el hardware**: minería, manufactura, transporte. Usar equipos que ya existen evita fabricar equipos nuevos.

Sobre este último punto conviene una precisión honesta. Al abaratar y facilitar la IA, es probable que la gente la use mucho más — es un fenómeno conocido: abaratar un recurso suele aumentar su consumo total. La respuesta de Swarmbly no es que esa demanda desaparezca, sino que **puede absorberse en hardware que ya existe**, sin construir centros de datos nuevos. Mientras haya capacidad ociosa disponible, el costo ambiental adicional de atenderla se acerca a cero.

11. Qué mostraron las primeras mediciones

Actualización, septiembre de 2026. Desde que se escribió esta sección se hizo una ronda de pruebas mucho más grande: 473 corridas, con cinco familias distintas de modelos pequeños. Salieron dos cosas. La primera es un costo: cuando a las dos formas de trabajar se les da el mismo espacio para escribir, repartir una tarea todavía pierde algo de calidad — alrededor de un 11 % en promedio—, aunque el programa que revisa los resultados se niega a darlo por definitivo, porque las dos formas de trabajar también escriben cantidades distintas, y eso por sí solo mueve la nota. La segunda es el resultado en torno al cual se construyó el diseño. En una tarea en la que un modelo pequeño tiene que leer muchos registros de un documento largo y responder preguntas sobre todos ellos, dejar que el modelo sólo lea, y que un programa corriente haga las cuentas, dio respuestas mucho mejores que pedirle al mismo modelo que lo hiciera todo de una vez. Y cuando el documento era lo bastante largo, leerlo por partes fue la única manera de que el modelo pequeño no se perdiera: de una sola vez, perdía el hilo. Es exactamente la situación para la que existe Swarmbly. Se midió con un conjunto pequeño de documentos y una sola familia de modelos,

y el relato completo está en la versión 2 del whitepaper. Lo que sigue es la ronda anterior, conservada tal como se escribió.

El proyecto ya se ha ejecutado contra modelos de lenguaje reales: modelos pequeños en un portátil. Se pusieron a prueba dos predicciones. **Ninguna de las dos pasó**, y las razones son distintas en cada caso. Las dos están aquí abajo.

La predicción central se puso a prueba como corresponde, y el umbral que el propio proyecto se fijó no se alcanzó. Todo el diseño descansa sobre una afirmación: cuanto más contexto compartido reciba cada participante, menos calidad se pierde al juntar las piezas. Antes de que existiera dato alguno, el proyecto escribió qué contaría como suficientemente bueno: la pérdida tenía que quedar **por debajo del 5 %** y —esto es lo importante— tenía que quedar por debajo del 5 % *incluso teniendo en cuenta la incertidumbre de la medición*, no solo en promedio.

La mejor medición disponible ahora, sobre un conjunto nuevo de 16 tareas reservadas de antemano para que ningún umbral pudiera ajustarse a ellas: partir una tarea en dos cuesta, en promedio, alrededor del **2.30 %** de la calidad. Eso parece un aprobado. No lo es. Como 16 tareas son pocas, la forma honesta de enunciar ese resultado es un rango y no una cifra única, y el rango va de **-2.05 % a +7.49 %**: su extremo superior queda por encima del 5 %. El umbral se escribió deliberadamente contra ese extremo superior, justamente para que un promedio favorable no pudiera sostener la afirmación por sí solo. Así que el veredicto es **no alcanzado** —le faltan 2.49 puntos en la cifra que cuenta— y el proyecto lo reporta en lugar de mover la portería que él mismo puso.

Lo genuinamente interesante está *por debajo* del promedio, y es un hallazgo nuevo. **La tarea típica no perdió nada en absoluto**: la tarea del medio, de las dieciséis, salió exactamente en **0.00 %**, y **11 de las 16 tareas quedaron en cero o mejor** —seis de ellas mejoraron y cinco salieron exactamente iguales—. El promedio no está simplemente arrastrado por las tareas caras: está *fabricado* por dos de ellas. Dos tareas de dieciséis costaron **+28.50 %** y **+23.08 %**, lo que entre las dos supone el **140 %** de todo el promedio. Si se quitan esas dos, las catorce restantes promedian **-1.06 %**: partir en dos sale ligeramente *mejor* que no partir. Así que la frase honesta no es «partir cuesta 2.3 %». Es: **en 11 de las 16 tareas, partir el trabajo en dos salió gratis; en dos de ellas salió caro.**

Sobre esas dos hay una pista que vale la pena nombrar. Son además las únicas dos tareas donde partir en **dos** salió peor que partir en **ocho** (+28.50 % frente a +13.57 %, y +23.08 % frente a +14.00 %). Si el problema fuera simplemente que fragmentar rompe la coherencia, más fragmentos deberían doler más, no menos. Así que la explicación más probable es que en esas dos tareas el corte se hizo en mal sitio: un problema de dónde se corta, y no un costo de cortar. Es una pista, no una afirmación puesta a prueba, y el proyecto no la ha puesto a prueba.

Nada de esto rescata el umbral. Sí cambia de qué trata el resultado: para un sistema que decide tarea por tarea si conviene partir, un costo que es gratis casi siempre y caro de vez en cuando resulta más útil que uno pequeño y uniforme, porque se puede aprender a esquivar los casos caros. Pero el listón que el proyecto se fijó a sí mismo no se ha superado, y el informe honesto lo dice. La ejecución también llevaba incorporada una prueba de cordura: un reparto deliberadamente excesivo que *tenía la obligación* de fallar, para que un instrumento averiado no pudiera aprobarlo todo en silencio. Falló como se le exigía, con un **16.23 %**, y con 15 de las 16 tareas peores que no partir en absoluto.

Un límite más que conviene decir con claridad: con tareas tan distintas entre sí, dieciséis de ellas no pueden producir un rango lo bastante estrecho como para caber por debajo de un

umbral del 5 %, sea cual sea el costo verdadero. Volver a poner a prueba el umbral exige antes un conjunto de tareas más amplio. Eso es un requisito previo, no una manera de obtener una respuesta más agradable.

Una versión anterior de este documento reportaba algo mucho mejor, y estaba equivocada. Decía que la predicción central se había cumplido, y que la calidad perdida bajaba de forma sostenida de alrededor del 24 % a alrededor del 14 % al subir el contexto compartido. Ese resultado queda **retirado**. El fallo no estaba en la arquitectura sino en el instrumento que la medía: estaba midiendo otra cosa, y de dos maneras distintas.

La primera es la más vergonzosa. La perilla que supuestamente se estaba girando —cuánto contexto compartido recibe cada participante— nunca giró de verdad. Todo paquete de trabajo tiene un piso: la tarea y sus instrucciones tienen que ir dentro, y eso ya cuesta algo antes de añadir una sola palabra de contexto compartido. Pon el presupuesto por debajo de ese piso y no queda sitio para contexto alguno, así que todos los ajustes producen exactamente el mismo paquete. En aquella primera ejecución, 13 de 96 mediciones quedaban por encima del piso; los dos ajustes que sostenían toda la tendencia descendente no tenían **ninguna**. De modo que la elegante curva descendente decía en realidad «los paquetes que no contienen nada rinden peor que los paquetes que contienen algo»: cierto, poco sorprendente, y no la afirmación que se estaba poniendo a prueba.

La segunda: la propia puntuación de calidad era injusta con la versión repartida. Ante el *mismísimo* texto, lo puntuaba 0.9375 cuando se le decía que estaba escrito de una pieza y 0.5000 cuando se le decía que venía en ocho — un castigo aparente del **46.7 %** sobre un texto que no había cambiado en nada. Una parte desconocida de aquella curva del 24 % al 14 % era simplemente esto.

Los dos fallos están corregidos, los dos tienen pruebas que los detectarán si alguna vez vuelven, y el programa ahora **se niega a publicar** una medición tomada por debajo del piso en lugar de imprimirla con una nota al pie.

La segunda predicción, el mapa de confianza, queda retirada del todo. El apartado 7 de este documento lo describe como aquello que la IA centralizada no puede darte: el sistema compara lo que dijeron varios modelos independientes y señala dónde discrepan. El mecanismo funciona exactamente como está diseñado: encuentra de verdad las discrepancias. La afirmación que falló es la útil: que allí donde los modelos coinciden, la respuesta tiende a ser *correcta*.

La primera medición no encontró relación alguna entre cuánto coincidían los modelos y lo buena que era la respuesta. En ese momento el veredicto justo era «sin demostrar, no desmentido», porque la prueba era débil: el corrector automático aceptó el 93 % de todo lo que vio, y así casi no quedaba nada contra lo que correlacionar. Así que el experimento se reconstruyó como correspondía, corrigiendo las respuestas contra una clave de respuestas correctas, y se ejecutó tres veces más.

Las tres ejecuciones se contradijeron entre sí. Medido como las probabilidades relativas de que una respuesta sea correcta, el acuerdo las multiplicó por **3.47** en la primera —un efecto real y útil si fuera cierto—. En la segunda las multiplicó por **0.26**, es decir, el acuerdo hacía una respuesta correcta *menos* probable. En la tercera, por **1.24**, que es indistinguible de no tener efecto alguno. Por encima de la línea, por debajo, y justo encima de ella, en la misma pregunta.

Eso no es una señal débil. Es ninguna señal, medida tres veces. De modo que el mapa de confianza queda **retirado**, y no meramente sin demostrar, y sale de la ronda de pruebas actual. Era, hasta estas mediciones, la característica de la que este proyecto estaba más

orgulloso — y lo honesto que se puede hacer con una idea preferida que falla es decirlo, en el mismo documento que la elogiaba. El mecanismo sigue ahí y sigue informando de dónde discreparon los modelos; lo que ya no puede hacer es decirte que el acuerdo significa corrección.

12. Lo que no se promete, y lo que todavía puede salir mal

Un proyecto que solo cuenta sus virtudes no merece confianza. Las dos mitades del relato honesto están reunidas aquí, en un solo sitio, en vez de repartidas por el documento.

Cuatro cosas que Swarmbly no afirma. **No es más rápido que una IA comercial** para quien ya tiene el equipo para usarla. **No tiene memoria infinita:** el límite de cuánto texto se puede procesar de una vez no desaparece, se traslada del servidor de una empresa a tu propio computador. Es un techo mucho más alto, y sube cuando mejoras tu equipo en vez de cuando cambias de plan de pago — pero es un techo. **No es un sistema secreto:** repartir la información en trozos hace más difícil que alguien la reconstruya, pero no es cifrado y sería deshonesto llamarlo así; para tareas realmente sensibles —salud, asuntos legales, datos personales— el diseño no las manda a computadores desconocidos, las mantiene dentro de un círculo cerrado o en tu propio equipo, como describe el apartado 9. Y **no ha demostrado todavía su beneficio ambiental:** el argumento es sólido, pero el proyecto se compromete a medirlo con un estándar público y publicar el resultado sea cual sea.

Dos riesgos que conviene nombrar. El técnico es que repartir una tarea y volver a juntarla cuesta calidad. Cuánta cuesta era una pregunta abierta; el proyecto construyó el instrumento que la mide y fijó el umbral de fracaso antes de tener dato alguno — y la mejor medición disponible cae del lado **equivocado** de ese umbral, a una escala, sobre 16 tareas (apartado 11). Es por poco, y 11 de las 16 tareas no pierden nada, pero «por poco» no es el estándar que el proyecto se escribió a sí mismo. El riesgo humano es el mayor: todo esto depende de que haya gente dispuesta a prestar su computador, y el cómputo voluntario lleva veinte años encogiéndose. Es bastante más probable que el proyecto fracase por falta de participantes que por un fallo de ingeniería. No sirve de nada tener el mejor protocolo del mundo si nadie se conecta.

Ese segundo riesgo es también la razón por la que existe este documento.

13. Por qué importa

Hay una asimetría en el centro de todo esto. El conocimiento para construir inteligencia artificial es público. El capital para operarla no lo es. Y esa diferencia —no un secreto, no una patente— es lo que concentra el control sobre una tecnología de propósito general.

Mientras tanto, el hardware capaz de hacer funcionar esa inteligencia está encendido y ocioso en cientos de millones de casas y oficinas. Unas 700000 máquinas ya están inscritas hoy en el cómputo voluntario, desde una comunidad que lleva veinte años encogiéndose. Esa cifra es un suelo, no un techo.

Swarmbly es una propuesta para conectar esos dos hechos. Si funciona, el resultado no es una manera más barata de comprar lo que ya se vende. Es capacidad de cómputo que **crece con el número de personas que participan** en lugar de crecer con el dinero disponible para construir — y ninguna empresa puede igualar esa curva, porque la capacidad de una

empresa está acotada por lo que puede financiar y la de una red está acotada por cuánta gente quiere que exista.

Conviene decirlo sin rodeos: esto es un intento de cambiar quién puede operar una tecnología de propósito general, no un intento de vender una versión con descuento de ella. Las primeras mediciones dicen que la idea no está obviamente equivocada, que es más de lo que la mayoría de las propuestas de este tamaño puede afirmar en su primer día.

Si no funciona, se sabrá, porque el proyecto publicó de antemano cómo comprobarlo y bajo qué condiciones lo daría por fallido.

Vale la pena intentarlo aun sabiendo que puede fracasar. Y vale la pena intentarlo a la vista de todos, donde cualquiera pueda revisarlo.

Glosario mínimo

Modelo de lenguaje — El programa que genera texto. Los hay grandes (los comerciales) y pequeños (los que caben en un computador normal).

Nodo — Un computador que participa en la red prestando capacidad.

Micro-tarea — Cada uno de los encargos en que se divide una petición grande.

Orquestador — El programa que corre en *tu* computador: decide si repartir, reparte, y vuelve a juntar las piezas.

Contrato común — La nota breve que acompaña a todos los encargos para que las piezas encajen entre sí.

Ensamblaje — Juntar las piezas devueltas en una respuesta única y coherente. El término viene de la genómica.

Alineamiento — Comparar varias respuestas a la misma tarea para ver dónde coinciden y dónde no.

Mapa de confianza — El informe que acompaña a la respuesta señalando qué partes son fiables y cuáles conviene revisar.

Enjambre de confianza — Una malla cerrada cuyos miembros figuran en una lista explícita, mantenida por un operador identificado, y en la que cada conexión demuestra la identidad de ambos extremos. El mismo protocolo que la red abierta, con la pertenencia restringida.

Círculo de privacidad (nivel) — A qué población de máquinas puede llegar una petición: la red abierta, un enjambre de confianza o tu máquina a solas. Es una cuestión distinta de lo sensible que sea el contenido, y ambas se deciden antes de enviar nada.

Este documento explica el proyecto para el público general. La descripción técnica completa, con sus fundamentos matemáticos, su especificación de protocolo y sus referencias, está en el artículo técnico que lo acompaña.

Versión en inglés: DIVULGACION_EN.md